Universidad del Valle de Guatemala
Facultad de Ingeniería
Programación Orientada Objetos
Segundo Ciclo, 2026
Catedrático: Erick Marroquín



Laboratorio 3

Estudiantes:
Emily Lucila Menchú Menchú - 20472
Edith Estefany Pérez Trujillo - 26952

Fecha: 9/10/2026

Laboratorio #3

Análisis

1. Requisitos funcionales, información a conservar, operaciones y condiciones

En general:

- Se debe indicar los días hábiles de la entrega cuando el cliente lo solicite
- Realizar cotizaciones dependiendo la categoría del paquete y datos de envío SIN registrar un envío
- Registrar envío al aceptar cotización, cobrar el monto completo y enviarlo a bodega.
- Cada vehículo debe ser identificado por una placa única y por su capacidad máxima de carga en kg, y debe verificarse si contiene refrigeración.
- Elegir un vehículo y asignar envíos en bodega.
- Asignar envío solo cuando: la suma de los pesos de los paquetes no supera la capacidad máxima del vehículo, asignar cargas refrigeradas en vehículos refrigerados, y cuando el envío no se encuentra asignado a otro despacho.
- Retirar envíos cuando se soliciten cuando estén en preparación
- Solamente debe existir un despacho sin cerrar en el vehículo.
- Al salir del despacho, cambiar el estado de los envíos a ruta.
- Si el despacho está vacío NO debe salir
- Registrar el resultado de envío ya sea entregado o entrega fallida (resultado de envío)
- Enviar un envío fallido a bodega puede asignarse a otro despacho
- Asignar un número de intentos permitidos
- Al superar los intentos permitidos, regresar envio al remitente y no volver a asignar
- Generar un cargo de Q15 cuando salga el despacho
- Cerrar el despacho cuando se registra el resultado y liberar vehiculo
- Reportes de envios de cada categoria y en cada estado , ingreso total (ingresos por registro, reintentos y categoria), despachos que no estén cerrados con vehiculo, peso cargado y capacidad disponible.
- Consultar los envios y vehiculos.
- Crear despachos, asignar y retirar envíos, hacer salir un despacho u registrar resultados de entregas

Información que el sistema debe conservar:

- Envío (común para todas las subclases): Código, remitente, destinatario, peso real en kg, tipo de destino (urbano o departamental), estado, número de intentos de entrega, ingreso por registro e ingreso por reintentos. Nota: el código debe de ser único y el peso real se cuenta para la capacidad del vehículo.
- Sobre: Peso máximo 0.5
- Paquete: Largo, ancho y alto. Se tiene que calcular el peso volumétrico
- Paquete frágil: Mismos valores que el paquete y el valor declarador que sirve para calcular el seguro.
- Carga Refrigerada: Temperatura requerida que debe de estar entre -25 y 8 grados Celsius.
- Vehículo: placa, capacidad máxima (kg) y si tiene refrigeración

- Despacho: Código, vehículo, estado, envíos asignados y el resultado de cada envío dentro de ese despacho.
- Ingresos: No se guardan en un contador, sino que cada envío guarda su ingreso por registro y su ingreso por reintentos y a la hora de hacer el reporte se van a sumar esos valores.
- Cotización: Tampoco se conserva, se calcula y se muestran el costo y el tiempo estimados para luego descartar la información.

Reglas de cobro, tiempos, vehículos y reintentos por categoría.

Categoría	**Datos y validaciones**	**Costo**	**Días (urbano / departamental)**	**Intentos**	**Vehículo**
Sobre	Peso ≤ 0.5 kg.	Q20 urbano o Q35 departamental y no importa el peso.	1 / 2	3	Cualquiera
Paquete	Largo, ancho y alto > 0.	Tarifa base: Q25 urbana y Q40 departamental. A la base se le suma Q8 por kilogramo facturable o fracción. Peso volumétrico = largo × ancho × alto / 5000. Peso facturable = mayor entre peso real y volumétrico. Kilos cobrados = peso facturable redondeado hacia arriba.	2 / 4	3	Cualquiera
Paquete frágil	Datos del paquete y valor declarado tiene que ser mayor a 0	Costo de un paquete más Q15 de embalaje + 3 % del valor declarado	Las del paquete + 1 (3 / 5)	2	Cualquiera
Carga refrigerada	Solo destino urbano. Temperatura entre −25 y 8 °C.	Tarifa base según destino + Q12 por kg de peso real + Q60 fijos por cadena de frío.	1	1	Solo vehículos refrigerados

Tomar en cuenta:

- Los datos iniciales (al menos dos envíos de cada categoría en bodega y al menos tres vehículos, uno refrigerado y uno no refrigerado) se cargan sin ingresos registrados.
- No instanciar objetos de clases abstractas
- Generar los cobros correctamente de acuerdo a la categoría de envío.
- Asegurar que el envío cambie de estado de acuerdo con su ubicación: en bodega (cuando esté registrado), asignado (despacho), ruta, entregado, fallido.
- Asignar un estado a despacho de preparación, cerrado
- Generar cargo de Q15 por intento adicional
- Información de los envíos: código, nombre de remitente, nombre de destinatario, peso real(kg), tipo de destino.
- Información sobre cada categoría de envío en cuanto a peso y precio de acuerdo con el destino.
- Evaluar la información de cada categoría de envío para asignar el cobro correcto
- El número de intentos de los envíos fallidos.
- Evaluar que un envío cuente con código único.
- Máximo de pesos en sobrecarga o carga refrigerada.

Condiciones:

- Un único código de envío
- No haber más de un despacho sin cerrar
- No dejar salir un despacho vacío
- No ingresar cobros vacíos
- No asignar nuevamente un envío al agotar sus intentos permitidos
- No alterar los ingresos en cotizaciones, asignaciones retiradas u operaciones rechazadas
- Una placa única por vehículo.
- Mantener disponible el menú hasta que el usuario decida salir
- Valores numéricos no positivos o fuera de lo permitido
- Asignaciones que excedan capacidad o no compatibles con vehículo
- Operaciones sobre envíos o despachos en estados no permitidos

2. Decisiones que dependen de la categoría del envío

Decisión	**Clase responsable**	**Justificación**
Validar datos según la categoría.	Método de registro y constructor de cada subclase.	El método detecta errores antes de crear el objeto y el constructor garantiza que siempre sea válido.

Calcular el costo del envío.	calcularCosto() en cada subclase.	Cada categoría aplica su fórmula y las subclases pueden reutilizar cálculos.
Calcular días de entrega.	calcularDiasEntrega() en cada subclase.	El tiempo depende de la categoría y el destino.
Verificar compatibilidad con el vehículo	esCompatibleCon() en Envio y CargaRefrigerada	Cada categoría determina el transporte adecuado
Definir el máximo de intentos.	getMaximoIntentos() en cada subclase.	Cada categoría tiene su límite de entregas (3, 3, 2 y 1).
Identificar la categoría.	getCategoria() en cada subclase.	Facilita la clasificación en reportes
Calcular tarifa base según destino	tarifaBase() en clase Envio	Reutiliza los montos Q25 y Q40 y se evita duplicar código.

3. Estados por lo que pasan envío y despacho
   Envío

Estado Origen	**Operación que genera transición**	**Estado destino**
(nuevo)	Se registra un nuevo envío	EN_BODEGA
EN_BODEGA	Asignar a un despacho	ASIGNADO
ASIGNADO	Retirar del despacho cuando el despacho sigue en preparación	EN_BODEGA
ASIGNADO	Cuando el despacho sale a ruta	EN_RUTA

EN_RUTA	Registrar resultado entregado	ENTREGADO
EN_RUTA	Registrar resultado fallido pero no se ha llegado al límite de intentos	EN_BODEGA
EN_RUTA	Registrar el resultado fallido pero los intentos ya llegaron al límite	DEVUELTO_AL_REMITENTE

Despacho

Estado origen	**Operacion**	**Estado destino**
(nuevo)	Se crea despacho si el vehículo existe y no tiene despachos abiertos	EN_PREPARACION
EN_PREPARACION	Cuando se añaden envíos que no superen la capacidad del vehículo o se retire, los envíos refrigerados estén en un vehículo refrigerado y cuando el envío no se asigne a otro despacho.	EN_PREPARACION
EN_PREPARACION	El despacho sale con al menos 1 envío	EN_RUTA
EN_RUTA	Registrar resultados, pero siguen habiendo envíos sin resultado.	EN_RUTA
EN_RUTA	Registrar el último resultado pendiente	CERRADO (vehículo queda libre)

4. Clases del programa:

Modelo:

- Envio (clase abstracta): Clase que establece atributos y métodos que todos lo tipos de envío deben de tener.
- Sobre: Clase que hereda de Envio
- Paquete: Extiende de envio. Clase que establece atributos y métodos que todos los tipos de paquetes deben de tener e incluso funciona como un tipo de paquete “normal”.
- Paquete frágil: Extiende de paquete e implementa sus atributos y lógica propia de un paquete frágil.

- Carga refrigerada: Extiende de envío e implementa sus atributos y lógica propia de una carga refrigerada.
- Vehiculo: Representa los vehículos, atributos y métodos.
- Despacho: clase que sirve para registrar los envios que salen de bodega y el vehículo que los lleva.
- SistemaEnvios: Guardar las colecciones, comprobar códigos únicos, coordinar operaciones y calcular reportes. Sirve para poner lógica relacionada con el modelo y no agregarle esa carga al Controlador.

Controlador:

- Controlador: maneja toda la lógica del menú y es un intermediario entre el modelo y la vista.

Vista:

- Vista: Se encarga de interactuar con el usuario, consultar datos y mostrar información.

Driver Program:

- Main: Es el driver program que permite inicializar el programa.

Enums:

- TipoDestino, EstadoEnvio, EstadoDespacho, ResultadoEntrega. Estas clases almacenan diferentes tipos de estados o resultados que se usarán como tipo de dato en varias clases.

Relaciones entre clases:

Herencia

- Sobre : Envio
- Paquete : Envio
- PaqueteFragil : Paquete
- CargaRefrigerada : Envio

Agregación

- SistemaEnvios : Envio
- SistemaEnvios : Vehiculo
- SistemaEnvios : Despacho
- Despacho : Vehiculo
- Despacho : Envio

Composición

- Controlador : SistemaEnvios
- Controlador : Vista

Dependencia

- Controlador : Sobre
- Controlador : Paquete
- Controlador : PaqueteFragil
- Controlador : CargaRefrigerada
- Main : Controlador
- Main : SistemaEnvios
- Main : Vista
- Envio : TipoDestino
- Envio : EstadoEnvio
- Despacho : EstadoDespacho
- Despacho : ResultadoEntrega

Tabla de atributos

Nota: Se agregaron atributos finales que representan datos que nunca cambian pero que se necesitan bastante a lo largo del sistema.

No.	Tipo de datos	Nombre	Visibilidad	Motivo	Clase
1	String	codigo	private	Identifica cada envío	Envio
2	String	remitente	private	Guarda el nombre del remitente	Envio
3	String	destinatario	private	Guarda el nombre del destinatario	Envio
4	double	pesoReal	private	Guarda el peso del envío	Envio
5	TipoDestino	destino	private	Indica el tipo de destino	Envio
6	EstadoEnvio	estado	private	Guarda el estado del envío	Envio
7	int	intentos	private	Cuenta los intentos de entrega	Envio
8	double	ingresoRegistro	private	Guarda el cobro inicial	Envio
9	double	ingresoReintentos	private	Guarda los cobros por reintentos	Envio
10	boolean	cobroRegistrado	private	Evita cobros repetidos	Envio

11	static final double	TARIFA_BASE_URBANA = 25	protected	Guarda la tarifa urbana	Envio
12	static final double	TARIFA_BASE_DEPARTAMENTAL = 40	protected	Guarda la tarifa para envíos a departamentos	Envio
13	static final double	CARGO_REINTENTO = 15	private	Guarda el costo por reintento	Envio
14	static final double	PESO_MAXIMO = 0.5	private	Define el peso máximo del sobre	Sobre
15	static final double	TARIFA_URBANA = 20	private	Guarda el precio urbano del sobre	Sobre
16	static final double	TARIFA_DEPARTAMENTAL = 35	private	Guarda el precio departamental del sobre	Sobre
17	double	largo	private	Guarda el largo del paquete	Paquete
18	double	ancho	private	Guarda el ancho del paquete	Paquete
19	double	alto	private	Guarda el alto del paquete	Paquete
20	static final double	DIVISOR_VOLUMETRICO = 5000	private	Se usa para calcular el peso volumétrico	Paquete
21	static final double	TARIFA_POR_KILO = 8	private	Guarda el costo por kilo	Paquete
22	double	valorDeclarado	private	Guarda el valor del paquete	PaqueteFragil
23	static final double	CARGO_EMBALAJE = 15	private	Guarda el costo del embalaje	PaqueteFragil
24	static final double	PORCENTAJE_SEGURO = 0.03	private	Guarda el porcentaje del seguro	PaqueteFragil
25	double	temperaturaRequerida	private	Guarda la temperatura necesaria	CargaRefrigerada

26	static final double	TEMPERATURA_MINIMA = -25	private	Define la temperatura mínima	CargaRefrigerada
27	static final double	TEMPERATURA_MAXIMA = 8	private	Define la temperatura máxima	CargaRefrigerada
28	static final double	TARIFA_POR_KG = 12	private	Guarda el costo por kilo	CargaRefrigerada
29	static final double	CARGO_CADENA_FRIO = 60	private	Guarda el costo de refrigeración	CargaRefrigerada
30	String	placa	private	Identifica cada vehículo	Vehiculo
31	double	capacidadMaxima	private	Guarda la capacidad del vehículo	Vehiculo
32	boolean	tieneRefrigeracion	private	Indica si tiene refrigeración	Vehiculo
33	String	codigo	private	Identifica cada despacho	Despacho
34	Vehiculo	vehiculo	private	Guarda el vehículo asignado	Despacho
35	EstadoDespacho	estado	private	Guarda el estado del despacho	Despacho
36	ArrayList<Envio>	envios	private	Guarda los envios	Despacho
	ArrayList<ResultadoEntrega>	resultados	private	Guarda los resultados de los envios	Despacho
37	ArrayList<Envio>	envios	private	Guarda los envíos registrados	SistemaEnvios
38	ArrayList<Vehiculo>	vehiculos	private	Guarda los vehículos registrados	SistemaEnvios
39	ArrayList<Despacho>	despachos	private	Guarda los despachos registrados	SistemaEnvios
40	SistemaEnvios	sistema	private	Permite usar las funciones del sistema	Controlador
41	Vista	vista	private	Permite mostrar y pedir datos	Controlador

42	Scanner	scanner	private	Permite leer datos del usuario	Vista

Tabla de métodos

No.	Tipo de dato	Nombre	Visibilidad	Parámetros	Motivo	Clase
1	-	Envio	protected	String codigo, String remitente, String destinatario, double pesoReal, TipoDestino destino	Inicializa los datos del envío	Envio
2	void	validarDatosComunes	public static	String codigo, String remitente, String destinatario, double pesoReal, TipoDestino destino	Valida los datos generales del envío	Envio
3	String	getCodigo	public	-	Devuelve el código del envío	Envio
4	String	getRemitente	public	-	Devuelve el nombre del remitente	Envio
5	String	getDestinatario	public	-	Devuelve el nombre del destinatario	Envio
6	double	getPesoReal	public	-	Devuelve el peso real	Envio
7	TipoDestino	getDestino	public	-	Devuelve el tipo de destino	Envio

8	boolean	esUrbano	public	-	Indica si el destino es urbano	Envio
9	EstadoEnvio	getEstado	public	-	Devuelve el estado del envío	Envio
10	int	getIntentos	public	-	Devuelve los intentos realizados	Envio
11	double	getIngresoRegistro	public	-	Devuelve el cobro inicial	Envio
12	double	getIngresoReintentos	public	-	Devuelve los cobros por reintentos	Envio
13	double	getIngresoTotal	public	-	Calcula el ingreso total del envío	Envio
14	double	calcularCosto	public abstract	-	Calcula el costo según la categoría	Envio
15	int	calcularDiasEntrega	public abstract	-	Calcula los días de entrega	Envio
16	int	getMaximoIntentos	public abstract	-	Devuelve el máximo de intentos	Envio
17	String	getCategoria	public abstract	-	Devuelve la categoría del envío	Envio
18	boolean	esCompatibleCon	public	Vehiculo vehiculo	Verifica si puede ir en el vehículo	Envio
19	double	tarifaBase	protected	-	Devuelve la tarifa según el destino	Envio
20	double	cargoPorReintento	protected	-	Devuelve el costo por reintento	Envio
21	boolean	tieneIntentosDisponibles	public	-	Verifica si quedan intentos	Envio
22	void	registrarCobroInicial	paquete	-	Registra el primer cobro	Envio

23	void	marcarAsignado	paquete	-	Cambia el envío a asignado	Envio
24	void	marcarRetirado	paquete	-	Devuelve el envío a bodega	Envio
25	void	iniciarIntento	paquete	-	Inicia otro intento de entrega	Envio
26	void	registrarResultado	paquete	boolean entregado	Guarda el resultado de entrega	Envio
27	String	toString	public	-	Muestra los datos del envío	Envio
28	-	Sobre	public	String codigo, String remitente, String destinatario, double pesoReal, TipoDestino destino	Crea un sobre con sus datos	Sobre
29	void	validarPeso	public static	double pesoReal	Valida el peso del sobre	Sobre
30	double	calcularCosto	public	-	Calcula el precio del sobre	Sobre
31	int	calcularDiasEntrega	public	-	Calcula los días de entrega	Sobre
32	int	getMaximoIntentos	public	-	Devuelve tres intentos	Sobre
33	String	getCategoria	public	-	Devuelve la categoría Sobre	Sobre

34	-	Paquete	public	String codigo, String remitente, String destinatario, double pesoReal, TipoDestino destino, double largo, double ancho, double alto	Crea un paquete con sus datos	Paquete
35	void	validarDimensiones	public static	double largo, double ancho, double alto	Valida las medidas del paquete	Paquete
36	double	getLargo	public	-	Devuelve el largo del paquete	Paquete
37	double	getAncho	public	-	Devuelve el ancho del paquete	Paquete
38	double	getAlto	public	-	Devuelve el alto del paquete	Paquete
39	double	calcularPesoVolumetrico	public	-	Calcula el peso volumétrico	Paquete
40	double	calcularPesoFacturable	public	-	Calcula el peso para cobrar	Paquete
41	long	calcularKilosCobrados	public	-	Redondea los kilos para cobrar	Paquete
42	double	calcularCosto	public	-	Calcula el precio del paquete	Paquete
43	int	calcularDiasEntrega	public	-	Calcula los días de entrega	Paquete
44	int	getMaximoIntentos	public	-	Devuelve tres intentos	Paquete
45	String	getCategoria	public	-	Devuelve la categoría Paquete	Paquete

46	String	toString	public	-	Muestra los datos del paquete	Paquete
47	-	PaqueteFragil	public	String codigo, String remitente, String destinatario, double pesoReal, TipoDestino destino, double largo, double ancho, double alto, double valorDeclarado	Crea un paquete frágil	PaqueteFragil
48	void	validarValorDeclarado	public static	double valorDeclarado	Valida el valor declarado	PaqueteFragil
49	double	getValorDeclarado	public	-	Devuelve el valor declarado	PaqueteFragil
50	double	calcularSeguro	public	-	Calcula el costo del seguro	PaqueteFragil
51	double	calcularCosto	public	-	Calcula el costo con seguro y embalaje	PaqueteFragil
52	int	calcularDiasEntrega	public	-	Agrega un día a la entrega	PaqueteFragil
53	int	getMaximoIntentos	public	-	Devuelve dos intentos	PaqueteFragil
54	String	getCategoria	public	-	Devuelve la categoría Paquete frágil	PaqueteFragil
55	String	toString	public	-	Muestra los datos del paquete frágil	PaqueteFragil

56	—	CargaRefrigerada	public	String codigo, String remitente, String destinatario, double pesoReal, TipoDestino destino, double temperaturaRequerida	Crea una carga refrigerada	CargaRefrigerada
57	void	validarRefrigeracion	public static	TipoDestino destino, double temperaturaRequerida	Valida el destino y la temperatura	CargaRefrigerada
58	double	getTemperaturaRequerida	public	-	Devuelve la temperatura requerida	CargaRefrigerada
59	double	calcularCosto	public	-	Calcula el costo de la carga	CargaRefrigerada
60	int	calcularDiasEntrega	public	-	Devuelve un día de entrega	CargaRefrigerada
61	int	getMaximoIntentos	public	-	Devuelve un intento	CargaRefrigerada
62	boolean	esCompatibleCon	public	Vehiculo vehiculo	Verifica si el vehículo tiene refrigeración	CargaRefrigerada
63	String	getCategoria	public	-	Devuelve la categoría Carga refrigerada	CargaRefrigerada
64	String	toString	public	-	Muestra los datos de la carga	CargaRefrigerada
65	-	Vehiculo	public	String placa, double capacidadMaxima, boolean tieneRefrigeracion	Crea un vehículo con sus datos	Vehiculo

66	void	validarDatosVehiculo	public static	String placa, double capacidadMaxima	Valida la placa y la capacidad	Vehiculo
67	String	getPlaca	public	-	Devuelve la placa del vehículo	Vehiculo
68	double	getCapacidadMaxima	public	-	Devuelve la capacidad máxima	Vehiculo
69	boolean	tieneRefrigeracion	public	-	Indica si tiene refrigeración	Vehiculo
70	String	toString	public	-	Muestra los datos del vehículo	Vehiculo
71	-	Despacho	public	String codigo, Vehiculo vehiculo	Crea un despacho en preparación	Despacho
72	void	validarDatosDespacho	public static	String codigo, Vehiculo vehiculo	Valida los datos del despacho	Despacho
73	String	getCodigo	public	-	Devuelve el código del despacho	Despacho
74	Vehiculo	getVehiculo	public	-	Devuelve el vehículo asignado	Despacho
75	EstadoDespacho	getEstado	public	-	Devuelve el estado del despacho	Despacho
76	ArrayList<Envio>	getEnvios	public	-	Devuelve los envíos asignados	Despacho
	int	buscarIndiceEnvio	public	String codigoEnvio	Busca el indice en la lista del envio que tiene el código pasado como parámetro	Despacho
77	double	calcularPesoCargado	public	-	Calcula el peso de los envíos	Despacho

78	double	calcularCapacidadDisponible	public	-	Calcula el espacio disponible	Despacho
79	void	validarAsignacion	public	Envio envio	Verifica si se puede asignar el envío	Despacho
80	void	agregarEnvio	public	Envio envio	Agrega un envío al despacho	Despacho
81	void	validarRetiro	public	String codigoEnvio	Verifica si se puede retirar el envío	Despacho
82	void	retirarEnvio	public	String codigoEnvio	Retira un envío del despacho	Despacho
83	void	validarSalida	public	-	Verifica si el despacho puede salir	Despacho
84	void	salir	public	-	Cambia el despacho a ruta	Despacho
85	void	validarResultado	public	String codigoEnvio	Verifica si se puede registrar el resultado	Despacho
86	void	registrarResultado	public	String codigoEnvio, boolean entregado	Guarda el resultado de entrega	Despacho
87	boolean	estaCerrado	public	-	Indica si el despacho está cerrado	Despacho
88	boolean	todosConResultado	private	-	Verifica si todos tienen resultado	Despacho
89	void	cerrar	private	-	Cierra el despacho al finalizar	Despacho
90	String	toString	public	-	Muestra los datos del despacho	Despacho
91	—	SistemaEnvios	public	-	Crea las listas del sistema	SistemaEnvios

92	void	validarCodigoEnvioDisponible	public	String codigo	Verifica que el código no esté repetido	SistemaEnvios
93	void	validarPlacaDisponible	public	String placa	Verifica que la placa no esté repetida	SistemaEnvios
94	void	validarCreacionDespacho	public	String codigo, String placa	Verifica si se puede crear el despacho	SistemaEnvios
95	void	validarAsignacionEnvio	public	String codigoDespacho, String codigoEnvio	Verifica si se puede asignar el envío	SistemaEnvios
96	void	validarRetiroEnvio	public	String codigoDespacho, String codigoEnvio	Verifica si se puede retirar el envío	SistemaEnvios
97	void	validarSalidaDespacho	public	String codigoDespacho	Verifica si el despacho puede salir	SistemaEnvios
98	void	validarResultadoEntrega	public	String codigoDespacho, String codigoEnvio	Verifica si se puede guardar el resultado	SistemaEnvios
99	void	registrarEnvio	public	Envio envio	Guarda el envío y registra el cobro	SistemaEnvios
100	void	precargarEnvio	public	Envio envio	Guarda un envío sin cobrar	SistemaEnvios
101	void	registrarVehiculo	public	Vehiculo vehiculo	Guarda un vehículo nuevo	SistemaEnvios
102	Despacho	crearDespacho	public	String codigo, String placa	Crea un nuevo despacho	SistemaEnvios
103	void	asignarEnvio	public	String codigoDespacho, String codigoEnvio	Asigna un envío al despacho	SistemaEnvios

104	void	retirarEnvio	public	String codigoDespacho, String codigoEnvio	Retira un envío del despacho	SistemaEnvios
105	void	hacerSalirDespacho	public	String codigoDespacho	Hace salir un despacho	SistemaEnvios
106	void	registrarResultadoEntrega	public	String codigoDespacho, String codigoEnvio, boolean entregado	Guarda el resultado de entrega	SistemaEnvios
107	ArrayList<Envio>	consultarEnvios	public	-	Devuelve los envíos registrados	SistemaEnvios
108	ArrayList<Vehiculo>	consultarVehiculos	public	-	Devuelve los vehículos registrados	SistemaEnvios
109	boolean	vehiculoEstaLibre	public	Vehiculo vehiculo	Verifica si el vehículo está libre	SistemaEnvios
110	String	contarPorCategoriaYEstado	public	-	Cuenta los envíos por categoría y estado	SistemaEnvios
111	double	ingresoPorRegistro	public	-	Suma los ingresos por registro	SistemaEnvios
112	double	ingresoPorReintentos	public	-	Suma los ingresos por reintentos	SistemaEnvios
113	double	ingresoTotal	public	-	Calcula el ingreso total	SistemaEnvios
114	String	ingresoPorCategoria	public	-	Calcula los ingresos por categoría	SistemaEnvios
115	ArrayList<Despacho>	despachosNoCerrados	public	-	Devuelve los despachos activos	SistemaEnvios

116	Envio	buscarEnvio	private	String codigo	Busca un envío por código	SistemaEnvios
117	Vehiculo	buscarVehiculo	private	String placa	Busca un vehículo por placa	SistemaEnvios
118	Despacho	buscarDespacho	private	String codigo	Busca un despacho por código	SistemaEnvios
119	-	Controlador	public	SistemaEnvios sistema, Vista vista	Conecta el sistema con la vista	Controlador
120	void	iniciar	public	-	Inicia el menú principal	Controlador
121	void	cargarDatosIniciales	private	-	Agrega los datos iniciales	Controlador
122	void	procesarOpcion	private	int opcion	Ejecuta la opción seleccionada	Controlador
123	void	cotizarYRegistrarEnvio	private	-	Cotiza y registra si el cliente acepta	Controlador
124	Envio	leerEnvio	private	-	Pide y valida los datos del envío	Controlador
125	void	registrarVehiculo	private	-	Pide los datos y registra el vehículo	Controlador
126	void	crearDespacho	private	-	Pide los datos y crea el despacho	Controlador
127	void	asignarEnvio	private	-	Solicita y asigna un envío	Controlador
128	void	retirarEnvio	private	-	Solicita y retira un envío	Controlador
129	void	hacerSalirDespacho	private	-	Solicita la salida del despacho	Controlador
130	void	registrarResultado	private	-	Registra el resultado de una entrega	Controlador

131	void	consultarEnvios	private	-	Muestra los envíos registrados	Controlador
132	void	consultarVehiculos	private	-	Muestra los vehículos registrados	Controlador
133	void	generarReporteEnvios	private	-	Muestra el reporte de envíos	Controlador
134	void	generarReporteIngresos	private	-	Muestra el reporte de ingresos	Controlador
135	void	generarReporteDespachos	private	-	Muestra el reporte de despachos	Controlador
136	—	Vista	public	-	Prepara la lectura de datos	Vista
137	int	mostrarMenu	public	-	Muestra las opciones del menú	Vista
138	String	leerTexto	public	String mensaje	Pide un texto al usuario	Vista
139	double	leerDecimal	public	String mensaje	Pide un número decimal	Vista
140	int	leerEntero	public	String mensaje	Pide un número entero	Vista
141	boolean	leerSiNo	public	String mensaje	Pide una respuesta de sí o no	Vista
142	TipoDestino	leerDestino	public	-	Pide el tipo de destino	Vista
143	void	mostrarMensaje	public	String mensaje	Muestra un mensaje	Vista
144	void	mostrarError	public	String mensaje	Muestra un mensaje de error	Vista
145	void	mostrarCotizacion	public	Envio envio	Muestra el costo y los días de entrega	Vista

146	void	mostrarReporteEnvios	public	String datos	Muestra el reporte de envíos	Vista
147	void	mostrarReporteIngresos	public	double registro, double reintentos, double total, String porCategoria	Muestra el reporte de ingresos	Vista
148	void	mostrarReporteDespachos	public	ArrayList<Despacho> despachos	Muestra el reporte de despachos	Vista
149	String	formatearMonto	public static	double monto	Muestra el monto con dos decimales	Vista
150	void	main	public static	String[] args	Inicia el programa	Main

Cómo reciben valores iniciales los objetos:

- Envio: Recibe código, remitente, destinatario, peso y destino. Inicia en EN_BODEGA, con 0 intentos y sin ingresos. El código se guarda en mayúsculas y sin espacios al inicio o final
- Sobre: Recibe los datos comunes de Envio y valida que el peso no supere 0.5 kg
- Paquete: Recibe los datos comunes de Envio y las dimensiones de largo, ancho y alto
- PaqueteFragil: Recibe los datos de Paquete y el valor declarado para calcular el seguro
- CargaRefrigerada: Recibe los datos comunes de Envio y la temperatura requerida, validando que sea urbana y esté entre −25 y 8 °C
- Vehiculo: Recibe placa, capacidad máxima y si tiene refrigeración. Sus datos no cambian después de crearlo
- Despacho: Recibe un código y un vehículo. Inicia en EN_PREPARACION y sin envíos asignados
- SistemaEnvios: No recibe parámetros. Inicia con tres colecciones vacías para envíos, vehículos y despachos
- Controlador: Recibe SistemaEnvios y Vista. Se encarga de cargar los datos iniciales sin generar ingresos
- Vista: No recibe parámetros. Inicializa el Scanner para leer los datos del usuario

Colaboración dentro de la jerarquía: Cada constructor llama primero a super(), así que la inicialización ocurre de arriba hacia abajo y cada clase se encarga únicamente de lo que le corresponde.

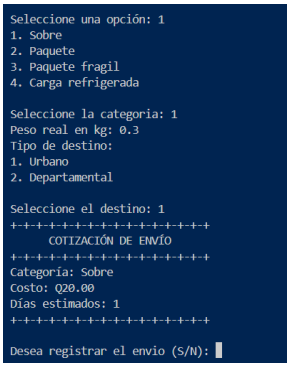
5. Justifique por qué el paquete frágil se relaciona con el paquete de la forma en que usted propone. Explique qué ganaría o perdería con una alternativa.

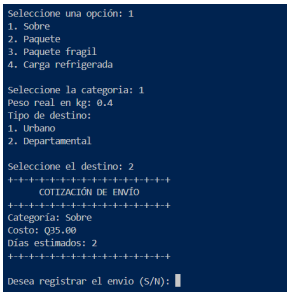
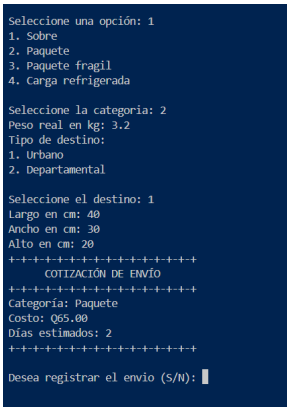
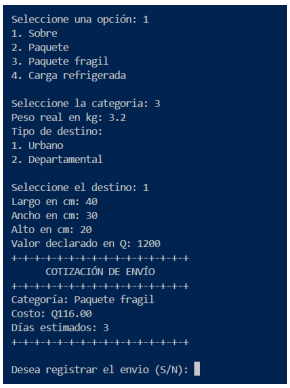
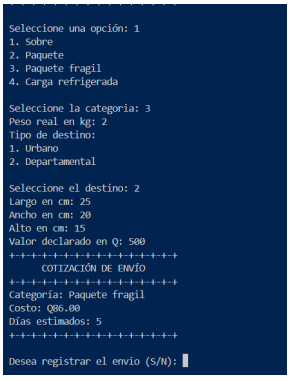
El paquete frágil se relaciona con el paquete por herencia, ya que el paquete frágil es un tipo especializado de paquete y comparte varios atributos y métodos, como la manera en la que se calcula el costo. Haciendo herencia, se puede ahorrar bastante código. La alternativa podría ser la composición, es decir, una clase PaqueteFragil que contiene un Paquete. En este caso habría menos acoplamiento y se podrían combinar otras clases, pero requeriría más código de delegación y complejidad e incluso podría ser menos semántico porque sabemos que paquete frágil es un paquete y no tiene un paquete. Por lo tanto, consideramos que la herencia es mejor en este caso aunque aumente el acoplamiento, ya que cualquier cambio en Paquete afectará a PaqueteFragil.

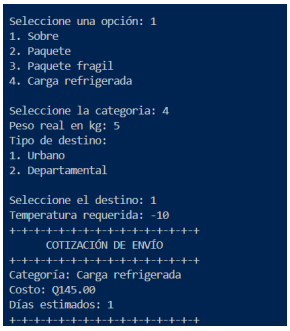
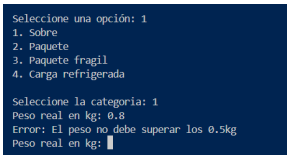
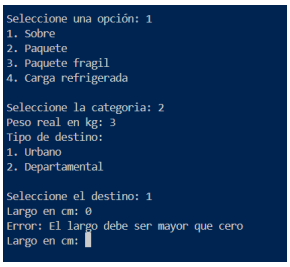
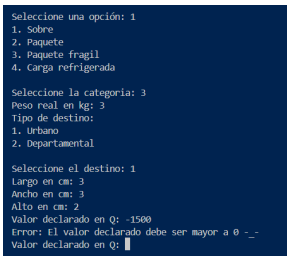
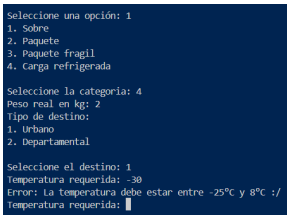
6. Explique qué clases tendría que crear o modificar para agregar una nueva categoría con reglas propias de costo, tiempo, vehículo y reintentos. Indique si los reportes o los despachos tendrían que cambiar.

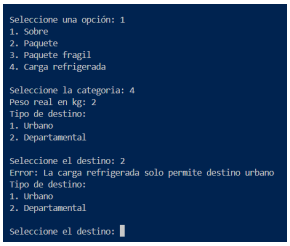
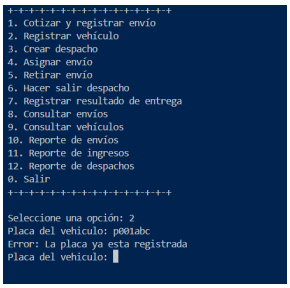
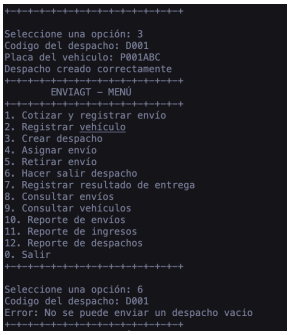
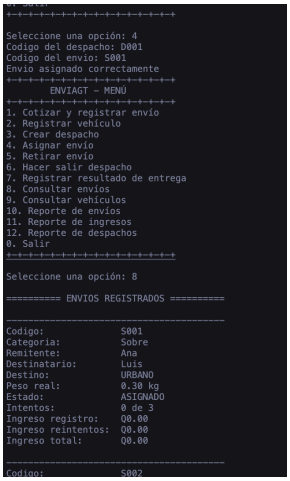
Si se creara una nueva categoría las clases que tendrían que cambiar serían el Controlador ya que se tienen que agregar la nueva opción en el menú y probablemente se cambien otras clases dependiendo de las características de esta nueva categoría, como un nuevo tipo de vehículo, etc., pero la mayoría de clases no se modificaría como: Envio, Despacho, SistemaEnvios, etc., ya que ya pueden manejar cualquier tipo de envio, esto devido a que la lógica personalizada de cada categoría ya se maneja dentro de la misma clase. Por lo tanto, los reportes y despachos no tienen que cambiar gracias al polimorfismo.

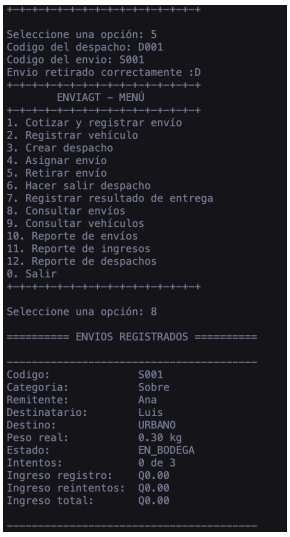
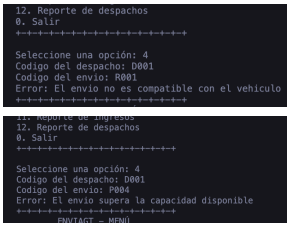
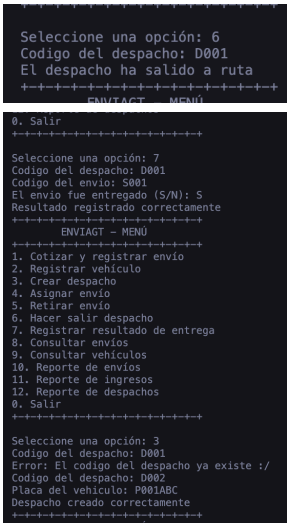
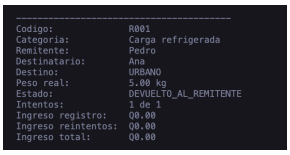
PRUEBAS

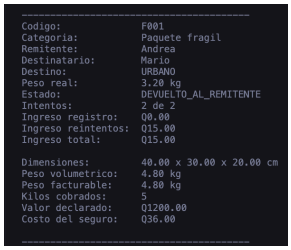
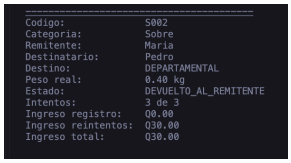
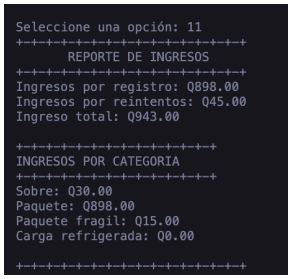
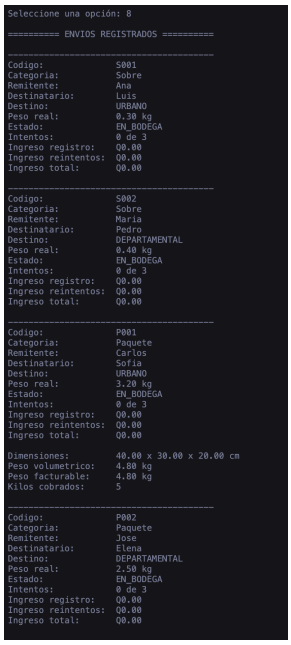
Prueba Cotizacion	Resultado esperado	Resultado obtenido	Evidencia
Sobre urbano (0.3kg, urbano)	Q20, 1 dia	Q20, 1 dia	

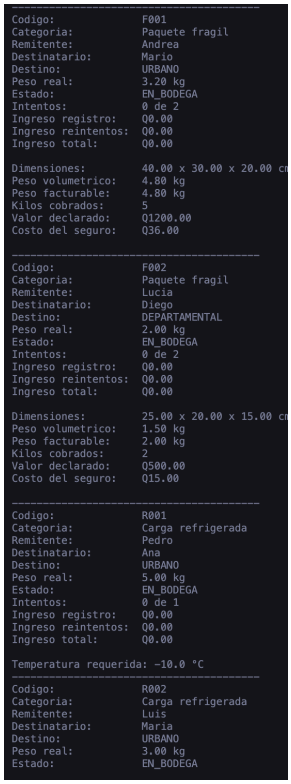
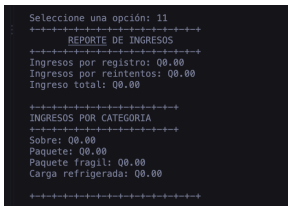
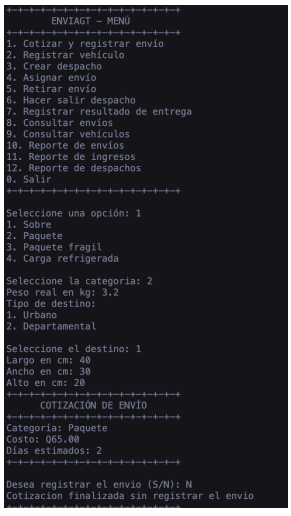
Sobre departamental (0.4kg, departamental)	Q35, 2 dias	Q35, 2 dias	
Paquete urbano (3.2 kg, 40 × 30 × 20 cm)	Q65.00, 2 días	Q65.00, 2 días	
Paquete frágil urbano (3.2 kg, 40 × 30 × 20 cm, valor Q1200)	Q116.00, 3 días	Q116.00, 3 días	
Paquete frágil departamental (2 kg, 25 × 20 × 15 cm, valor Q500)	Q86.00, 5 días	Q86.00, 5 días	

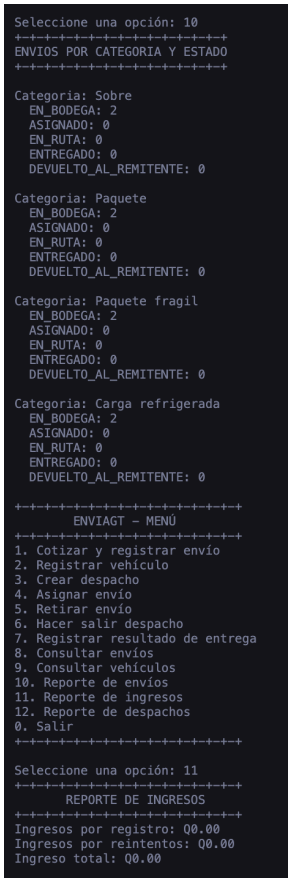
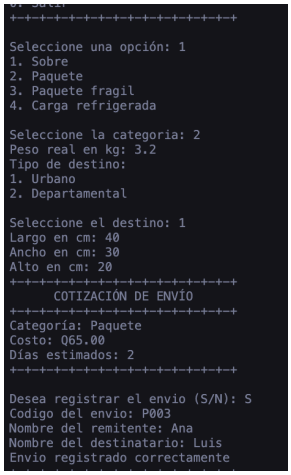
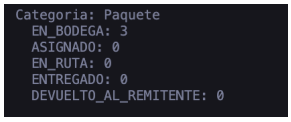
Carga refrigerada (5 kg, urbano, −10 °C)	Q145.00, 1 día	Q145.00, 1 día	
Prueba validacion	Resultado esperado	Resultado obtenido	Evidencia
Registrar sobre de 0.8 kg	Rechazar registros y pedir volver a ingresar el valor	Registro rechazado y solicitud de datos válidos para continuar	
Registrar paquete con dimensión de 0 cm	Rechazar registros y pedir volver a ingresar el valor	Registro rechazado y solicitud de datos válidos para continuar	
Registrar paquete frágil con valor declarado negativo	Rechazar datos inválidos y pedir volver a ingresar el valor	Registro rechazado y solicitud de datos válidos para continuar	
Registrar carga refrigerada a −30 °C	Rechazar registros y pedir volver a ingresar el valor	Registro rechazado y solicitud de datos válidos para continuar	

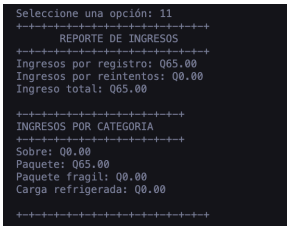
Registrar carga refrigerada con destino departamental	Rechazar registros y pedir volver a ingresar el valor	Registro rechazado y solicitud de datos válidos para continuar	
Registrar vehículo con placa repetida	Mostrar errores y no parar el programa	Registro rechazado y solicitud de datos válidos para continuar	
Despachos			
Crear despacho vacío e intentar hacerlo salir	Rechazar salida	Se rechazó la salida	
Asignar y retirar `S001` de un despacho	Cambiar de asignado a bodega	Cambió de asignado a bodega al retirar el envio.	

			
Asignar `R001` a vehículo sin refrigeración y probar una carga que exceda la capacidad	Rechazar asignaciones	Se rechazaron las asignaciones	
Hacer salir un despacho y registrar todos los resultado	Pasar a ruta y al terminar cerrar despacho y liberar vehiculo	Pasó a ruta, al terminar, se registraron los resultados y se liberó el vehículo, luego se pudo crear un nuevo despacho con el mismo vehículo	
Registrar fallos consecutivos de un sobre, un paquete frágil y una carga refrigerada	Máximo de 3, 2 y 1 intentos, respectivamente. Al agotarlos, quedan `DEVUELTO AL REMITENTE`. Se cobran Q15 por cada intento adicional.	Al llegar al maximo de intentos se dovolvió al remitente y se cobraron Q15 por cada intento adicional.	

			  
Consultar envíos e ingresos al iniciar	8 envíos en bodega, 2 por categoría, ingresos Q0.00.	8 registros, 2 por categoría y los ingresos son 0.	

			 
Cotizar sin registrar	No hay registro	La cotización no se registró; los envíos siguen siendo los mismos que los datos iniciales y no hay ingresos registrados.	

			
Registrar envio	Envío registrado en EN_BODEGA	Se registra un envío. En el reporte de envíos vemos que hay un nuevo registro en bodega y en el reporte de ingresos se ve reflejado el ingreso por el envío.	 

			 Seleccione una opción: 11 REPORTE DE INGRESOS Ingresos por registro: Q65.00 Ingresos por reintentos: Q0.00 Ingreso total: Q65.00 INGRESOS POR CATEGORIA Sobre: Q0.00 Paquete: Q65.00 Paquete fragil: Q0.00 Carga refrigerada: Q0.00